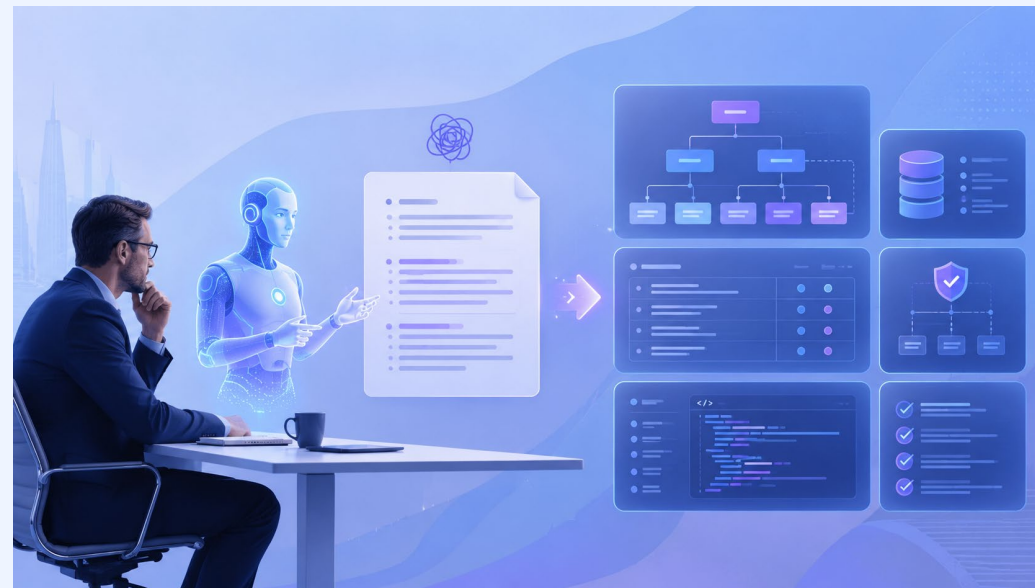


AI-помощник аналитика \ QA \ поддержка и корпоративная база знаний

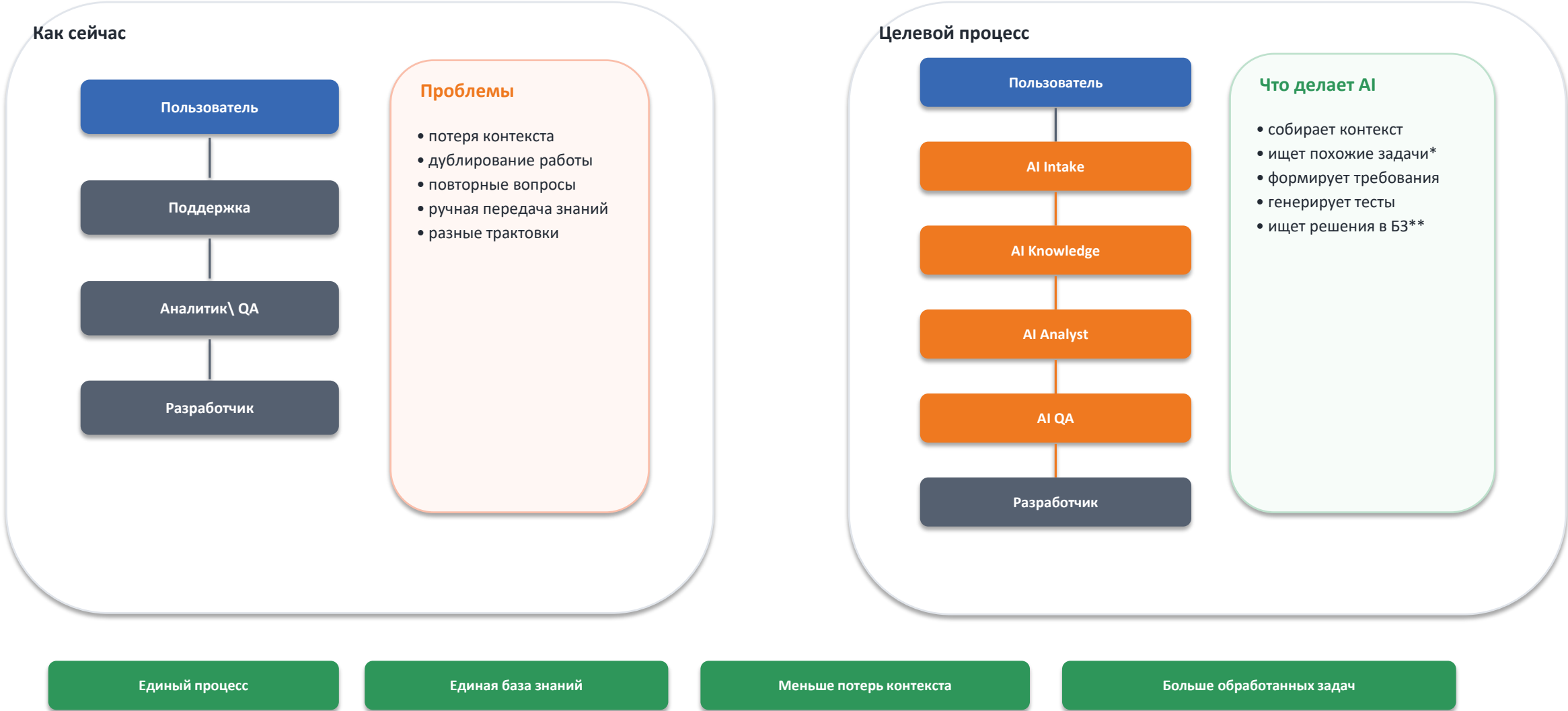
Что решаем

проверка задач, уточнения, ТЗ, база знаний



AI как единое звено между Аналитиком, QA и Поддержкой

AI не заменяет сотрудников — он убирает рутину и сохраняет контекст между ролями



Что внутри презентации

1. Проблема и целевая модель

простым языком + практические шаги

2. 4 задачи заказчика и как их реализовать

простым языком + практические шаги

3. База знаний: структура, процессы, RAG

простым языком + практические шаги

4. n8n и AI-агенты: как собрать MVP

простым языком + практические шаги

5. Пошаговый план внедрения

простым языком + практические шаги

Проблема: задачи и знания живут в хаосе

Система нужна не ради AI, а чтобы убрать ручную рутину и потери знаний

Как сейчас

- задачи в, Jira, email
- ТЗ неполные или разного формата
- аналитик\1 и 2 линия вручную задают вопросы
- похожие задачи ищут по памяти
- FAQ и решения теряются в чатах
- новые сотрудники долго входят в проект
- Дублирующие задачи от разных заказчиков в итоге трата времени (разные сотрудники разбирают одну проблему по разному не знаю друг о друге)

Что болит

- много уточнений
- задержки разработки
- повторные ошибки
- конфликт версий документов
- зависимость от конкретных людей
- нет единого источника правды

Что должно быть

- единый процесс приема задач
- AI проверяет полноту
- база знаний помогает отвечать
- ТЗ создается по шаблону
- человек подтверждает результат
- знания обновляются автоматически

Рост пропускной способности команды

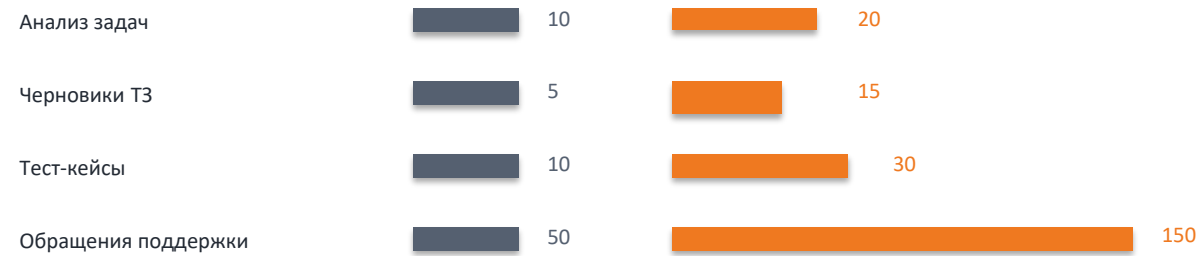
AI берет первичную рутину, а люди остаются владельцами решений и качества

Сейчас: ручная работа

- Анализ задач — Аналитик
- Поиск решений — Поддержка
- Подготовка ТЗ — Аналитик
- Тест-кейсы — QA
- Инструкции — Поддержка

После AI: автоматизация рутины

- Первичный анализ — AI
- Поиск решений — AI
- Черновик ТЗ — AI
- Тест-кейсы — AI
- Инструкции — AI



Основная идея: AI не заменяет сотрудников — он позволяет обрабатывать больше задач тем же составом команды

Единая первичная оценка для TOA, TSI и Поддержки

AI Intake — единая точка входа для всех обращений первой и второй линии



Результат: единый стандарт обработки, меньше ошибочных задач, быстрее маршрутизация и меньше нагрузки на TOA

AI для QA и Поддержки

Черновая работа автоматизируется, а сотрудники контролируют качество и сложные случаи

AI QA: сокращение ручного тест-дизайна

Test Cases

Smoke Tests

Regression Tests

Negative Scenarios

Edge Cases

QA выполняет review,
дополняет сценарии и
подтверждает качество.

Эффект: быстрее тесты + выше покрытие

AI Support: инструкции, FAQ и мультиязычность

FAQ

Инструкции пользователей

Инструкции поддержки

Release Notes

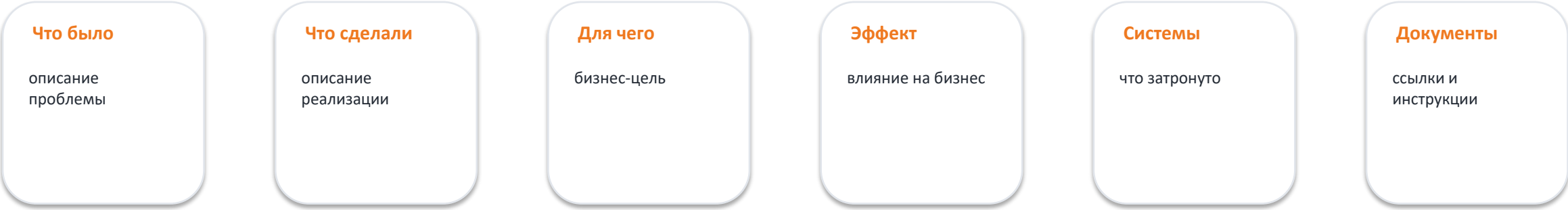
Ответы на обращения

Мультиязычность:
RU / EN / TR / DE / AR

Эффект: поддержка быстрее отвечает пользователям

PDF Knowledge Report по каждой задаче

После закрытия задачи результат автоматически превращается в знание компании



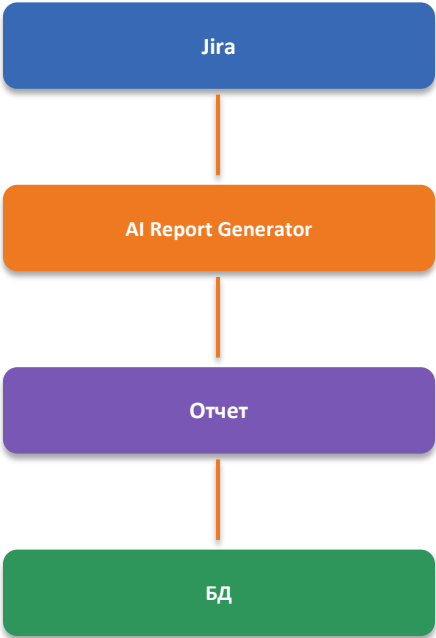
Основная идея: каждая закрытая задача пополняет корпоративную память и помогает будущим проектам

Корпоративная память компании

Новые поля в задаче + отчет + база данных создают историю решений и бизнес-эффекта

Новые поля в Jira / задаче*

- Бизнес-проблема
- Цель
- Что сделано
- Польза для бизнеса
- Влияние на процессы
- Затронутые системы
- Ответственный
- Ссылки на документы



Метаданные в PostgreSQL*

task_id
project
created_date
closed_date
responsible
business_value
business_impact
pdf_link
kb_article_link

4 задачи, которые нужно решить

Каждая задача закрывается отдельным AI-процессом, но все процессы работают на общей базе знаний

1. Первичная оценка задачи

AI проверяет БТ/ТЗ, задает вопросы и переводит бизнес-запрос на технический язык

Предварительная оценка: от 4 недель*

2. Черновик документации

AI готовит ТЗ, user story, acceptance criteria, тест-кейсы, API draft и т.д

Предварительная оценка: 5-8 недель*

3. Анализ полноты и мусора

AI отсеивает неописанные задачи, дубли, противоречия и слабые формулировки

Предварительная оценка: 5-8 недель*

4. База знаний проектов **

Единая память: системы, шаблоны, FAQ, глоссарий, примеры, регламенты

Предварительная оценка: от 8 недель*

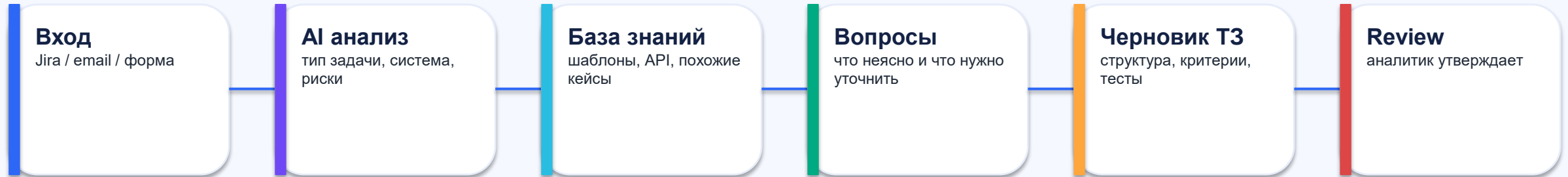
AI + n8n + Knowledge Base | концепция внедрения

* Вся оценка является предварительной, при условии работы над параллельными проектами, временные рамки реализации могут менять как в большую так и в меньшую сторону, так как задействованы разные проекты и сотрудники, так же зависит скорость предоставления данных от отделов. Оценка не означает фактически потраченное время на реализацию

** База знаний сложна в реализации, была попытка сделать для ДМС – не вышло. Необходимо всю информацию привести к единому стилю и формату. Потребуется много работы от разных команд

Целевой процесс: от сырого запроса до готового ТЗ

Человек остается владельцем решения, AI ускоряет подготовку и проверку



Важно

AI не заменяет аналитика. Он делает черновую работу: ищет контекст, задает вопросы, приводит задачу к стандарту, предлагает документацию. Аналитик принимает итоговое решение.

Кейс TSI.

Проблема

- >40% обращений — не технические инциденты
- Корень: коммуникационные разрывы между отделами и низкая информированность пользователей
- Инструкции уже существуют, но не находятся или не применяются

Что делает AI

Запрос → бот ищет решение в БЗ (напр., Инструкция №588)

Решение найдено → мгновенная выдача инструкции/ссылки

Решение не найдено → авто-создание заявки → маршрутизация (2-я линия → ТОА или сразу в ТОА для анализа)

Что получает аналитик

- 📉 Снижение нагрузки
- ⚡ Ускорение реакции: пользователь сразу видит, что делать
- 🎯 Фокус команды только на реальных инцидентах и доработках продукта



<https://confluence.fstravel.com/pages/viewpage.action?pageId=110694588>

<https://jira-new.fstravel.com/secure/Dashboard.jspa?selectPageId=11318>

Кейс 1. Первичная оценка задач

Задача: превратить сырой бизнес-запрос в понятную техническую постановку

Что приходит

“Нужно добавить уведомления клиентам”

Что делает AI

Определяет систему, процесс, роли, каналы уведомлений, события, ограничения и недостающие данные.

Что получает аналитик

Список уточняющих вопросов + переформулированный технический запрос.



[Пример практики n8n AI workflow: docs.n8n.io/advanced-ai/intro-tutorial/](https://docs.n8n.io/advanced-ai/intro-tutorial/)

Кейс 1. Как реализовать пошагово

Минимальная реализация первичной оценки задач

1. Канал входа

Создать форму в Jira/YouTrack/and: описание, проект, приоритет, система, желаемый результат.

2. Триггер n8n

При новой задаче n8n забирает текст, автора, проект, вложения и запускает workflow.

3. Классификатор AI

LLM определяет тип: баг, фича, изменение процесса, интеграция, отчет, доступ.

4. Проверка полноты

AI прогоняет задачу по чек-листу: роли, события, ограничения, данные, интеграции, SLA.

5. Вопросы

AI формирует вопросы только по недостающим пунктам, без лишнего шума.

6. Техническая формулировка

AI переписывает бизнес-запрос как техническую постановку для аналитика/разработчика.

Кейс 1. Пример: как AI улучшает задачу

До AI задача выглядит коротко, после AI - понятно что делать

Было

“Нужно сделать уведомления клиентам о заказе.”

Проблемы:

- непонятно кому отправлять
- непонятно когда отправлять
- нет канала
- нет шаблона
- нет ошибок и повторов
- нет требований к логированию

Стало

“При изменении статуса заказа система отправляет клиенту уведомление по email и push. События: создан, оплачен, передан в доставку, доставлен, отменен. Нужны шаблоны сообщений, логирование, retry 3 раза, хранение истории отправок и API для просмотра статуса.”

Кейс 1. Что использовать для MVP и production

Под каждый сценарий - простой старт и промышленный вариант

MVP

- n8n workflow
- Jira trigger
- LLM prompt “проверка задачи”
- чек-лист полноты
- результат комментарием в задачу

Цель: быстро снизить ручные уточнения.

Production

- отдельный Task Intake API
- RBAC и аудит
- интеграция с Jira
- RAG по базе знаний
- правила по проектам
- хранение результатов проверок

Цель: единый стандарт приема задач.

[n8n AI Agents](#)

[n8n AI workflow tutorial](#)

[n8n AI workflow templates](#)

Кейс 2. AI готовит черновик документации

Задача: ускорить подготовку ТЗ, но оставить контроль за аналитиком

Что генерирует AI

- краткое описание
- цель задачи
- scope / out of scope
- роли пользователей
- бизнес-правила
- acceptance criteria
- edge cases
- тест-кейсы
- API draft

Откуда берет контекст

- похожие задачи
- шаблоны ТЗ
- API docs
- FAQ
- глоссарий
- правила проекта
- прошлые решения
- регламенты

Что делает человек

- проверяет смысл
- убирает лишнее
- подтверждает решения
- добавляет ограничения
- согласует с бизнесом
- утверждает итог

Кейс 2. Как реализовать пошагово

1. Шаблон ТЗ

Утвердить единый шаблон: цель, контекст, требования, API, критерии приемки, тесты.

2. Библиотека примеров

Собрать 20-50 хороших ТЗ по ключевым типам задач.

3. Prompt templates

Создать отдельные промпты: user story, acceptance criteria, API draft, test cases.

4. RAG контекст

Перед генерацией AI получает похожие документы и правила проекта.

5. Черновик

AI создает draft и помечает места, где не хватает данных.

6. Review

Аналитик правит документ и нажимает “утвердить/отправить на доработку”.

Кейс 2. Рекомендуемый шаблон ТЗ

1. Контекст

Зачем задача нужна бизнесу, какая проблема решается

2. Требования

Что должно работать, какие сценарии и ограничения

3. Интеграции

Системы, API, события, форматы данных

4. Критерии приемки

Как понять, что задача выполнена правильно

5. Тесты и риски

Edge cases, негативные сценарии, риски данных

Кейс 2. MVP и production для документации

Начинаем с шаблонов, потом подключаем RAG и контроль качества

MVP

- Confluence
- 3-5 шаблонов ТЗ
- n8n кнопка “создать черновик”
- LLM генерирует markdown
- аналитик вручную проверяет

Результат: меньше времени на первый черновик.

Production

- версионирование документов
- автоссылки на похожие задачи
- mandatory review
- качество документа по чек-листу
- публикация в KB после утверждения

Результат: единый стандарт документации.

[Atlassian Confluence knowledge base guide](#)

[Wiki.js](#)

[Open WebUI Knowledge docs](#)

Кейс 3. Анализ полноты и отсечение мусора

AI должен не только писать, но и защищать процесс от плохих задач

Что считаем мусором

- “сделать красиво”
- “поправить как раньше”
- нет цели
- нет системы
- нет результата
- нет владельца
- нет данных
- есть только скрин без описания

Что проверяет AI

- понятна ли цель
- есть ли пользователь
- есть ли сценарий
- есть ли ограничения
- есть ли данные
- есть ли интеграции
- есть ли критерии приемки

Что выдает AI

- статус качества
- score 0-100
- список пробелов
- вопросы автору
- предупреждения
- рекомендация: принять / уточнить / отклонить

Кейс 3. Как реализовать пошагово

1. Матрица качества

Определить обязательные поля: цель, система, процесс, данные, результат, критерии.

2. Правила отсеечения

Если нет цели/владельца/системы - задача возвращается на уточнение.

3. AI score

LLM оценивает полноту по шкале: 0-40 плохо, 40-70 нужно уточнить, 70+ можно брать.

4. Дубли

RAG ищет похожие задачи и предупреждает: "возможно, уже есть решение".

5. Комментарий

n8n публикует результат в задачу: вопросы, риски, рекомендации.

6. Обучение

Используем исправленные задачи как хорошие примеры для базы знаний.

Кейс 3. Чек-лист полноты задачи

Бизнес-цель

понятно зачем делаем

Пользователи

кто участвует в сценарии

Система

какой сервис/модуль затронут

Сценарии

основной и альтернативные пути

Данные

какие поля, форматы, источники

Интеграции

API, события, очереди, внешние системы

Ограничения

SLA, безопасность, права доступа

Приемка

как проверяем готовность

Кейс 4. База знаний проектов *

Что берем из Platrum

КБЗ должна содержать инструкции, регламенты, шаблоны, FAQ, лучшие практики; важно определить цели, структуру, доступы, ответственных.

Что берем из ECM Journal

Эффективность базы повышают участие сотрудников, классификация, теги, фильтры, аналитика, обучение и поэтапное внедрение.

Что это значит для AI

Сначала приводим знания в порядок, затем подключаем RAG и AI-агентов. Иначе AI будет красиво генерировать ошибки.



[Platrum: стратегия создания корпоративной базы знаний](#)

[ECM Journal: как создать мощную базу знаний](#)

Кейс 4. Структура базы знаний

Структура должна быть одинаковой для проектов, чтобы AI мог находить и сравнивать информацию

Рекомендуемая структура

Проект

- 01. Обзор системы
- 02. Архитектура
- 03. API и интеграции
- 04. Бизнес-процессы
- 05. ТЗ и user stories
- 06. FAQ
- 07. Ошибки и решения
- 08. Шаблоны
- 09. Глоссарий
- 10. Регламенты

Зачем это нужно

AI легче искать знания, если документы имеют одинаковые разделы и понятные названия.

Что важно

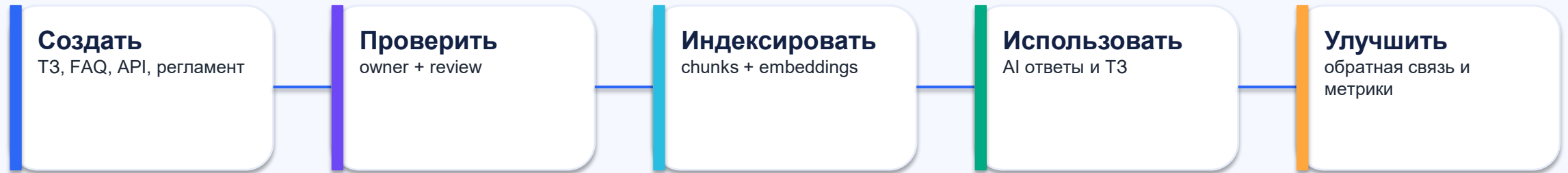
У каждого раздела должен быть владелец, дата актуальности, статус и ссылка на источник.

Правило

Нет владельца и даты актуализации - документ считается слабым источником для AI.

Как будет работать база знаний

Жизненный цикл знаний: создать - проверить - использовать - обновить



Принцип

База знаний живет только если встроена в ежедневную работу: AI берет из нее контекст, аналитики обновляют документы, система показывает пробелы и устаревшие материалы.

Кейс 4. Как реализовать базу знаний пошагово

Пошаговый план нужен, чтобы не получить “еще одну папку с файлами”

1. Инвентаризация

Собрать документы: ТЗ, API, FAQ, регламенты, схемы, старые задачи.

2. Очистка

Удалить дубли, пометить устаревшее, выделить актуальные источники.

3. Таксономия

Утвердить разделы, теги, проекты, типы документов, глоссарий.

4. Шаблоны

Создать шаблоны: ТЗ, API, FAQ, регламент, incident, integration.

5. Владельцы

Назначить owner для каждого проекта и раздела.

6. Индексация

Подключить RAG: chunking, embeddings, vector DB, обновление индекса.

7. Доступы

Настроить роли: аналитики, разработчики, бизнес, внешние подрядчики.

8. Метрики

Считать поисковые запросы, пробелы, устаревшие документы, полезность ответов.

RAG простыми словами

Это способ дать AI доступ к знаниям компании без “угадывания”

Обычный AI

Отвечает из общей модели. Может не знать внутренние системы, API и правила компании.

[Open WebUI RAG docs](#)

RAG

Сначала ищет нужные документы в базе знаний, потом отвечает на основе найденного контекста.

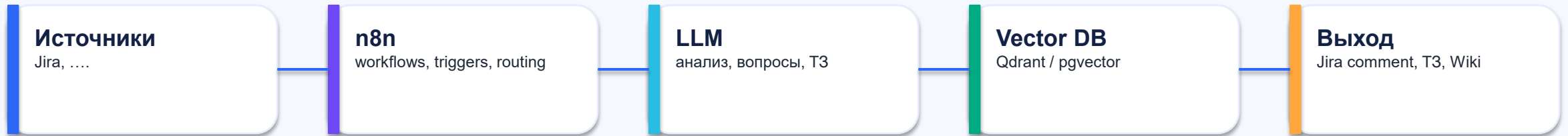
[Qdrant search docs](#)

Польза

Меньше галлюцинаций, больше ссылок на источники, проще проверить ответ.

Техническая архитектура MVP

Минимальный рабочий контур на n8n + LLM + база знаний



1. Task Intake

Новая задача запускает workflow. n8n забирает текст, автора, проект и вложения.

2. AI + RAG

AI ищет контекст в базе знаний, проверяет полноту и генерирует результат.

3. Human Review

Аналитик подтверждает, правит или отправляет задачу на уточнение.

n8n: какие workflows нужны

n8n выступает оркестратором: соединяет системы, AI и людей

Workflow 1

Проверка новой задачи

Workflow 2

Генерация уточняющих вопросов

Workflow 3

Создание черновика ТЗ

Workflow 4

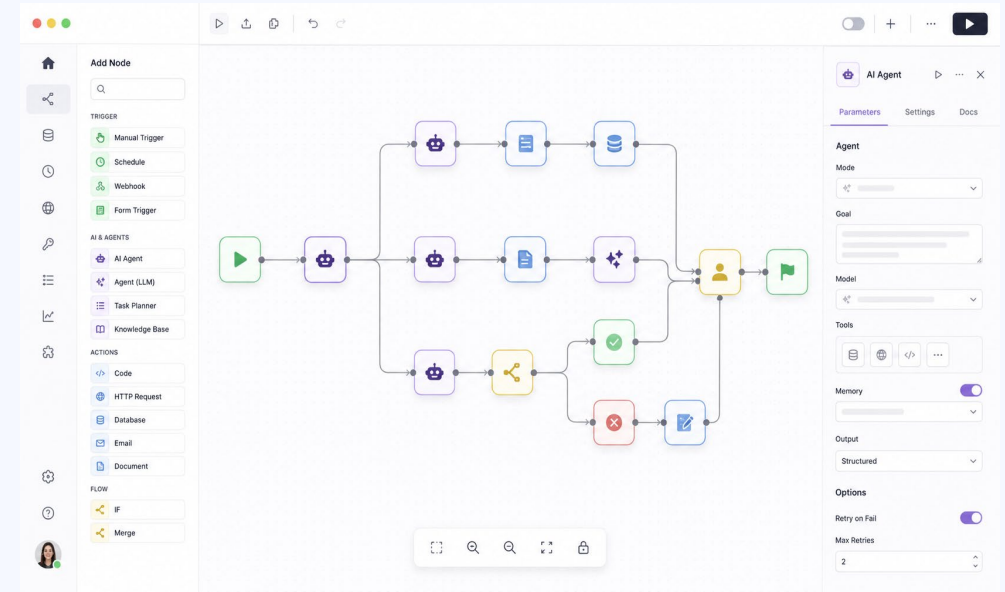
Индексация базы знаний

Workflow 5

Поиск похожих задач

Workflow 6

Контроль устаревших документов



[n8n AI agents](#)

[n8n AI workflows](#)

AI-агенты: роли в системе

Не один “бот на все”, а набор специализированных помощников

AI Intake Agent

принимает задачу, классифицирует, проверяет обязательные поля

AI Analyst Agent

задает вопросы, описывает бизнес-логику, формирует user story

AI Documentation Agent

создает черновики ТЗ, FAQ, API draft, тест-кейсы

AI Knowledge Agent

ищет похожие задачи, источники, правила, глоссарий

AI Reviewer Agent

проверяет полноту, противоречия, риски, соответствие шаблону

AI QA Agent

готовит тестовые сценарии и edge cases

Экономический эффект

Все метрики фиксируются до и после внедрения

Подготовка ТЗ:

6ч → 2ч

Тест кейсы:

4ч → 20м

Ответ поддержки:

30м → 5м

Поиск знаний:

30ч → 5м

Пошаговый план внедрения:

Начинаем с простого, но сразу закладываем основу для production

Диагностика

выбрать 1-2 проекта, собрать типовые задачи, определить боли и метрики

Стандарты

шаблоны ТЗ, чек-лист полноты, глоссарий, правила приемки задач

MVP AI Intake

n8n workflow: вход задачи, проверка, вопросы, комментарий в Jira

База знаний

структура, загрузка документов, владельцы, теги, индексация

AI документация

генерация черновика ТЗ, acceptance criteria, тест-кейсов

Пилот и улучшение

сбор метрик, feedback, доработка prompts, решение о масштабировании

Что должно быть готово после MVP

MVP должен показать пользу, а не просто “поговорить с AI”

Рабочий AI intake

новая задача автоматически проверяется и получает вопросы

Шаблон ТЗ

единый формат для аналитиков

Мини-база знаний

20-50 документов: FAQ, ТЗ, API, регламенты

RAG поиск

AI ищет похожие кейсы и источники

Метрики

сколько задач улучшено, сколько вопросов закрыто

Пилотная группа

аналитики реально используют процесс

Продвинутый уровень: куда развивать систему

Важно начать просто, но проектировать с запасом

Multi-agent platform

разные агенты для аналитики, архитектуры, QA и ревью

Knowledge Graph

связи между системами, API, бизнес-процессами и задачами

Auto decomposition

AI предлагает декомпозицию эпиков на задачи и подзадачи

Impact analysis

AI показывает какие системы и процессы затронет изменение

Quality dashboard

дашборд качества задач, документации и базы знаний

Self-improving KB

AI находит пробелы, устаревшие документы и предлагает обновления

Метрики успеха

Чтобы проект не был “AI ради AI”, заранее фиксируем измеримые показатели

Время на первичный анализ

до / после внедрения

Количество уточнений

сколько вопросов снимает AI

Доля задач без доработки

сколько задач проходят intake

Время подготовки ТЗ

среднее время на черновик

Повторное использование знаний

сколько раз AI нашел полезный источник

Качество базы знаний

устаревшие документы, дубли, пробелы

Adoption

сколько аналитиков реально используют систему

Ошибки в разработке

сколько дефектов из-за плохого ТЗ

Риски и как их снизить*

Главный риск - автоматизировать хаос. Поэтому сначала структура, потом AI

Плохие данные

сначала инвентаризация, очистка и владельцы знаний

Галлюцинации AI

RAG, ссылки на источники, human review

Сопrotивление сотрудников

пилот, обучение, простые сценарии, быстрые победы

Утечки данных

RBAC, локальные модели, маскирование ПДн

Слишком сложный старт

делать MVP на одном проекте, не пытаться внедрить все сразу

Устаревшая база знаний

метрики актуальности, owner, регулярная ревизия